

Experimental Speed Synchronization of a Dual-BLDC Tracked Differential-Drive Vehicle Using a GA-Tuned Bilateral Cross-Coupled Controller on a Dual-MCU Architecture

[TODO: Author One], [TODO: Author Two], and [TODO: Author Three]

Abstract—Differential-drive and skid-steer ground vehicles steer by the relative speed of their left and right drives, so any uncommanded difference between the two track speeds appears directly as heading drift and path error. The problem is aggravated by low-cost brushless DC (BLDC) motors, whose unit-to-unit variation in winding resistance and back-EMF constant makes two nominally identical drives accelerate and cruise differently. This paper presents and experimentally validates a distributed speed-synchronization controller for a tracked skid-steer vehicle driven by two mismatched three-phase BLDC “auto-rickshaw” motors. Each motor is regulated by its own microcontroller running a PID loop with velocity and acceleration feedforward; a bilateral cross-coupled term, evaluated identically on both nodes from a dedicated 1 ms inter-microcontroller speed exchange, drives the inter-motor speed difference toward zero, while a master node distributes commands and logs data over an RS-485 bus. All gains are obtained offline by a genetic algorithm against an identified motor model and then deployed on the vehicle. Relative to an independent-PID (synchronization-off) baseline, the controller reduces the peak transient inter-motor speed error by 79%, its RMS value by 73%, and the accumulated straight-line heading drift by 80%; under a single-track load disturbance it cuts the sustained speed difference and the heading excursion by about 65%. The result is a transparent, low-cost synchronization scheme that needs no runtime plant model and runs on commodity microcontrollers.

Index Terms—Brushless DC (BLDC) motor, speed synchronization, cross-coupled control, differential drive, skid-steer, genetic algorithm, PID control, feedforward, embedded systems, dual-microcontroller.

I. INTRODUCTION

DIFFERENTIAL-DRIVE and skid-steer ground vehicles steer by controlling the relative speed of their left and right drives. Any uncommanded difference between the two wheel/track speeds therefore appears directly as heading drift and path error, even when each drive tracks its own speed reference well. On a tracked vehicle the effect is unforgiving: a small, persistent speed imbalance between the two tracks integrates over distance into a growing lateral deviation and yaw error, so a platform commanded to run straight curves away from its intended path. For a remotely operated vehicle expected to hold a heading without continuous operator correction, this drift is both a usability and a safety concern, and

it cannot be removed by tuning each drive’s own speed loop more tightly — the two loops must be coordinated.

The problem is aggravated when the two drives are *not identical*. Low-cost brushless DC (BLDC) motors — such as the auto-rickshaw motors used in this work — exhibit unit-to-unit variation in winding resistance and back-EMF constant, so at equal command the two drives accelerate and cruise differently. This mismatch is a persistent, structured disturbance rather than random noise: it biases one track faster than the other for the entire run, producing exactly the steady heading drift that independent per-motor control cannot see. Rejecting it in real time is the role of an inter-drive *speed-synchronization* controller that feeds the speed difference back into both drives [1], [2].

Cross-coupled speed synchronization is itself well established, but the existing evidence does not transfer cleanly to this setting. Most multi-motor synchronization studies are validated in simulation or on matched permanent-magnet synchronous machines (PMSMs) mounted on a bench under a single controller; the more advanced robust schemes (sliding-mode, model-predictive, or consensus control) achieve excellent synchronization but rely on accurate plant models, state observers, or reliable real-time inter-agent communication that are hard to justify on a low-cost, distributed embedded target [3]–[5]. Metaheuristic PID tuning, in turn, is mature but is almost always applied to a *single* motor in simulation [6], [7]. Few works report hardware results for a real, deliberately mismatched-BLDC differential-drive vehicle realized with one microcontroller per motor exchanging speed over a dedicated inter-MCU link [8]–[10]. This paper targets precisely that combination.

Contributions. This paper presents:

- a *distributed dual-MCU* implementation of bilateral cross-coupled speed synchronization for two deliberately mismatched 3-phase BLDC drives, in which the two motor microcontrollers exchange speed over a dedicated 1 ms inter-MCU link while a master microcontroller distributes commands and logs data over an RS-485 bus;
- *experimental validation on a real tracked (skid-steer) differential-drive vehicle* powered by auto-rickshaw BLDC motors;
- a controller combining per-motor PID with velocity/acceleration feedforward, static-mismatch decoupling, and a differential-acceleration feedforward, whose gains

Manuscript submitted [TODO: date], 2026. [funding / acknowledgment if any]

The authors are with [TODO: Department, Institution, City, Country] (e-mail: [TODO: corresponding@email]).

are obtained by *offline genetic-algorithm tuning against an identified motor model*;

- a quantified reduction in inter-wheel speed error and straight-line heading drift versus an independent-PID (synchronization-off) baseline, with repeatability statistics.

The remainder of the paper is organized as follows. Section II reviews synchronization structures and metaheuristic PID tuning. Section III describes the hardware platform. Section IV presents the controller design, including the offline GA tuning. Section V details the dual-MCU implementation. Section VI reports the experimental results, and Section VII concludes.

II. RELATED WORK

A. Multi-Motor Synchronization Structures

Coordinated multi-motor control is conventionally organized along two axes: the *control structure* that routes inter-motor information and the *control algorithm* that acts on it [3], [11]. Structurally, schemes divide into *non-coupling* forms—parallel control, where each motor independently tracks a shared command, and master–slave control, where followers track the master’s output—and *coupling* forms that feed a synchronization error back between drives [12], [13]. The non-coupling forms are simple but fragile: with no cross-feedback, a disturbance on one motor cannot be corrected by the others, and master–slave control is asymmetric, so a disturbance on the master propagates to every follower while a follower’s disturbance is never returned [3], [11].

Coupling control originates with Koren’s cross-coupled bi-axial controller, which computes a contour (synchronization) error from both axes and injects a weighted correction into *both* loops simultaneously [1]; Feng *et al.* first carried this idea to a differential-drive mobile robot, coupling the two wheel loops through the vehicle’s orientation error [2]. Subsequent variants generalize the coupling rule to n motors—improved cross-coupling with a softened reference [14], relative coupling that decouples synchronization from tracking through separate gains [15], deviation coupling driven by the average and sub-average speeds [16], and ring coupling in which each motor synchronizes only with its neighbour [17]—while electronic line-shafting couples drives through a software “virtual shaft” [12], [18], [19], a scheme with a long industrial lineage in paper machines [20]. A unifying observation, confirmed across these formulations, is that for the *two-motor* case they all reduce to the same symmetric bilateral law $\beta_i = K(\omega_i - \omega_j)$: relative, deviation, and ring coupling collapse to it when $n=2$, and the passive-decomposition “shape” coordinate is exactly $\omega_1 - \omega_2$ [21]. Our controller therefore adopts the *bilateral cross-coupled* structure—equivalent to Koren’s law for two drives and the canonical choice for a two-motor differential platform—rather than a higher-order coupling topology that offers no benefit at $n=2$.

B. Beyond PID: Robust and Advanced Synchronization

A large and active body of work pushes synchronization beyond linear PID. Sliding-mode control (SMC) dominates:

global fast terminal SMC combined with cross-coupling for dual-motor systems [22], [23], non-singular fast terminal SMC with deviation coupling [24], adaptive SMC with ring coupling [17], and cross-coupled fractional-order SMC for linear motors [25]. Other strands add disturbance-observer-based fuzzy backstepping over a multi-agent graph [26], unified finite-control-set MPC with a first-order SMC inner loop [4], fixed-time leader–following consensus [5], approximation-free prescribed-performance control [27], decoupled robust control built on mechanical-parameter identification [28], artificial potential fields with sliding-mode variable structure [29], and hierarchical H_2/H_∞ coordination for dual-BLDC steer-by-wire [30]. These methods can be highly effective—e.g., a unified MPC/SMC scheme drives peak dual-PMSM synchronization error from over 100 r/min to below 1 r/min under unbalanced load [4], and an identification-based robust law reports an order-of-magnitude lower synchronization RMS than PID on a gantry [28]. The cost, however, is model dependence and implementation burden: accurate plant models, observer or sliding-surface design, Lyapunov/finite-time gain conditions, or real-time inter-agent communication—the last of which the consensus literature itself flags as fragile to delay and packet loss [5]. A recent review that benchmarks FOC, MPC, and SMC under matched conditions finds that PID-based FOC in fact attains the best *steady-state* tracking and synchronization accuracy, with the modern methods winning mainly in transient adjustment time [3]; the fuzzy alternatives surveyed in [13] likewise trade interpretability for an expert-designed rule base. We therefore retain a transparent PID-plus-feedforward law—deployable on a low-cost, distributed microcontroller target without a plant model or observer—and obtain performance through systematic offline optimization and a fast bilateral synchronization loop.

C. Genetic-Algorithm and Metaheuristic PID Tuning

Because the value of a transparent PID hinges on its gains, metaheuristic tuning is well established. Genetic algorithms (GAs) have tuned PID controllers for BLDC speed control [6], [31], [32] and position control [7], for DC drives [33], and—most recently—in combination with a sliding-mode observer for sensorless PMSM speed control [34]; broader surveys and statistical comparisons frame the metaheuristic PID-tuning field and its open problems [35], [36], and methodological studies guide the choice of GA crossover and mutation rates [37]. The reported gains are large: GA tuning reduces BLDC step overshoot from 54% to below 1% versus Ziegler–Nichols [7] and, on real BLDC hardware, cuts overshoot by 73% and steady-state error by 98% [6], while a spec-driven weighted-sum objective is shown to outperform generic IAE/ISE criteria [33]. Two limitations recur. First, almost all of this work tunes a *single* motor in *simulation*; where a metaheuristic is applied to a wheeled robot it optimizes one trajectory-tracking loop rather than inter-motor synchronization [38], and the few multi-motor cases remain simulation-only [39]. Second, the optimization target is typically a single PID. We extend GA tuning to a *coupled two-motor* problem—jointly evolving each motor’s PID, the synchronization controller, and

the feedforward terms against a multi-scenario fitness—and deploy the optimized gains on hardware.

D. Differential-Drive Coordination and Embedded Realization

For wheeled and tracked vehicles the synchronization problem is concrete: unequal wheel speeds produce heading drift, so the two-side independent drive is the dominant scheme and its coordination is a core control problem [2], [13]. Reported approaches span PI wheel-speed synchronization on a differential-drive robot [40], hybrid-metaheuristic PID with a multi-motor synchronization procedure on a four-wheel drive [9], a synchronous-error compensator for wheeled cross-coupled axles [41], master–slave versus cross-coupling on a two-motor bench [10], and dedicated dual-BLDC controller hardware for a differential-drive rover [8]. On the implementation side, low-cost microcontroller BLDC control with sensorless back-EMF commutation [42], STM32-based speed control with cascaded speed/current loops [43], and single-chip cross-coupling multi-motor drives [44] demonstrate on-device feasibility. Our platform goes further on the architectural axis: control is *distributed*, with one microcontroller per motor linked by a custom RS-485 master–slave (Modbus-style) bus whose timing and framing we characterize in prior work [45]. Each cited effort supplies one ingredient—a platform, a tuning method, a communication bus, or an embedded loop—but not their combination, and vehicle-level studies seldom report quantitative inter-wheel synchronization.

E. Gap Addressed by This Work

To our knowledge, no prior work combines (i) GA-tuned *bilateral cross-coupled* PID synchronization, (ii) two *deliberately mismatched*, low-cost BLDC drives, (iii) a *distributed* one-microcontroller-per-motor realization over a custom Modbus/RS-485 bus, and (iv) *experimental validation on a real tracked differential-drive vehicle*. Synchronization studies are predominantly simulation-only or use matched PMSMs on benches; metaheuristic PID tuning is predominantly single-motor; and embedded or vehicle-level realizations rarely report quantitative inter-wheel synchronization performance. This paper targets exactly that combination.

III. SYSTEM DESCRIPTION AND HARDWARE PLATFORM

The test platform (Fig. 1) is a tracked, skid-steer differential-drive vehicle. Its two rubber tracks are driven independently by two 3-phase brushless DC (BLDC) “auto-rickshaw” motors, each through a 20:1 reduction gearbox. Steering is produced by a difference between the two track speeds, so during straight-line motion the synchronization controller’s task is to hold the two motor speeds equal despite the motors’ electrical and load mismatch. The vehicle carries a 6-degree-of-freedom manipulator arm; the platform as tested weighs 120 kg (a 75 kg carrier plus a 45 kg arm), so the tracks must coordinate under a substantial and movable load. Control is *distributed*: each motor is closed-loop controlled by its own microcontroller; the two motor microcontrollers exchange their measured speeds

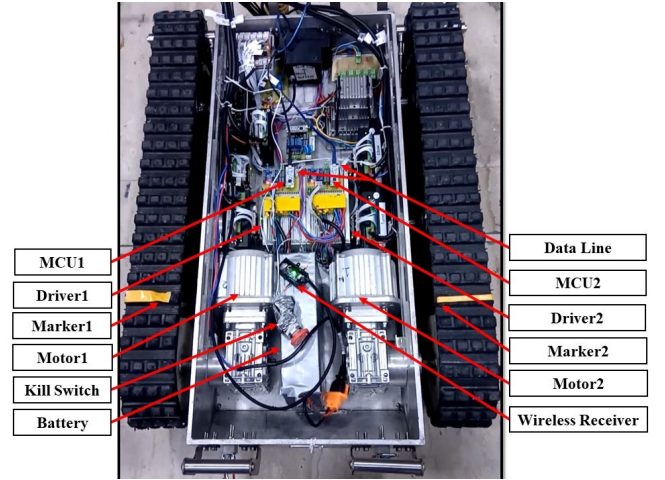


Fig. 1: Annotated top view of the tracked skid-steer differential-drive ROV. Visible subsystems: the two geared 3-phase BLDC motors (Motor 1/Motor 2) at center with their off-the-shelf controllers (Driver 1/Driver 2) and per-motor microcontrollers (MCU 1/MCU 2); the dedicated inter-MCU data line; the shared Li-ion battery; the manual kill switch; the wireless receiver; and the yellow track markers (Marker 1/Marker 2) used to visualize the inter-track speed difference during a run. The STM32F446RE master node and the RS-485 bus serving the wider vehicle are also on board.

over a dedicated point-to-point link for synchronization, while a separate master microcontroller distributes the operator’s speed command and logs data over a shared serial bus. All experiments are run outdoors on open ground.

A. Mechanical and Electrical Architecture

The chassis (1.5 m × 0.75 m × 0.5 m) carries two rubber-belt tracks on a rigid running gear of two idlers per side with a rear drive sprocket. The track gauge—the lateral distance between the two track centerlines—is $B = 1.2$ m, and the ground-contact length is $L = 1.0$ m; this $L/B \approx 0.83$ ratio sets the skid-steer turning behaviour. Each drive sprocket has radius $r = 0.20$ m.

Each track is driven by a 3-phase BLDC auto-rickshaw motor (rated 48 V, 750 W, 15.6 A, four pole pairs) through its 20:1 gearbox. The motor shaft is the controlled variable: the speed loop regulates motor-shaft speed up to a commanded ceiling of 3000 rpm, which the gearbox converts to about 150 rpm at the sprocket and a track ground speed of $v = \omega r \approx 3.14 \text{ m s}^{-1}$ (11.3 km h⁻¹) at the top of the range. This gearing matters for the synchronization problem: a relative track-speed difference Δv produces a heading rate $\dot{\psi} = \Delta v/B$, so even a 1% speed mismatch at full speed yields $\dot{\psi} \approx 1.5^\circ \text{ s}^{-1}$ —roughly 15° of heading drift in ten seconds—which no amount of single-loop tuning on either motor can remove.

Each motor is driven by a ready-made, sensed e-rickshaw BLDC controller (48 V/750 W class, current-limited near 30 A, with an internal under-voltage cutoff near 41.5 V). The controller performs the six-step trapezoidal commutation internally from the motor’s Hall sensors; the microcontroller

commands it with a single throttle (speed) line and a direction line, and is not involved in commutation. Both motor-controller channels are identical. Power is a shared 13-cell (13S) Li-ion battery (48.1 V nominal, 15 A h, ≈ 720 W h): the two controllers draw directly from the 48 V bus, while the microcontrollers are supplied through a step-down (buck) converter. Safety is provided by a chassis-mounted manual emergency-stop that disconnects all power, and by a 20 A fuse in each motor circuit (between the 15.6 A rated current and the controller's ≈ 30 A limit).

B. Distributed Control and Communication Architecture

Control is partitioned across three microcontrollers (Fig. 2) on two distinct communication paths. Each motor is regulated by its own STM32G431KB microcontroller (Nucleo-32 board, 170 MHz) running a local speed PID. The two motor microcontrollers are linked by a *dedicated point-to-point UART channel* over which each transmits its measured motor speed to the other every 1 ms using DMA; this fast, private link carries the data the synchronization loop needs. Separately, a master STM32F446RE microcontroller is connected to both motor nodes over an RS-485 bus (MAX485 transceivers, differential twisted-pair) layered on UART at 115 200 baud with custom framing, DMA-driven transfers, and a per-packet CRC; this bus distributes the operator's speed command and gathers data for logging. The two motor nodes are, in fact, two of the slaves on this larger multi-node bus, which also serves the vehicle's manipulator and sensors; its design, addressing, and timing are characterized in our prior work [45].

Over the RS-485 bus the master converts the operator's motion command into a speed reference expressed as a percentage of maximum speed (0–100%), broadcasts that reference to both motor nodes, and polls each node for its measured speed, which it records for logging. Straight-line motion corresponds to an equal reference to both motors; a turn corresponds to differential references.

The synchronization itself is computed *locally and symmetrically on the two motor nodes*, not on the master. Each motor node maps the broadcast percentage to an RPM set-point, runs its own speed PID on the error between that set-point and its measured motor-shaft speed, and—using the other motor's speed received over the dedicated inter-MCU link every 1 ms—evaluates the bilateral cross-coupled synchronization correction. Both nodes run the identical synchronization law with the same coefficients, so both compute the same correction value; each then applies one half of it with opposite sign ($+\frac{1}{2}$ on one motor, $-\frac{1}{2}$ on the other). This is a direct, distributed realization of the symmetric bilateral law $\beta_i = K(\omega_i - \omega_j)$ to which all two-motor coupling structures reduce (Section II): the law is evaluated redundantly on both nodes and split $\pm\frac{1}{2}$ across them. The master's role is confined to command distribution, bus sequencing, and logging; it does not close the synchronization loop.

C. Sensing and Measurement

Speed feedback is taken from each motor's three Hall-sensor lines, read by the corresponding microcontroller through timer

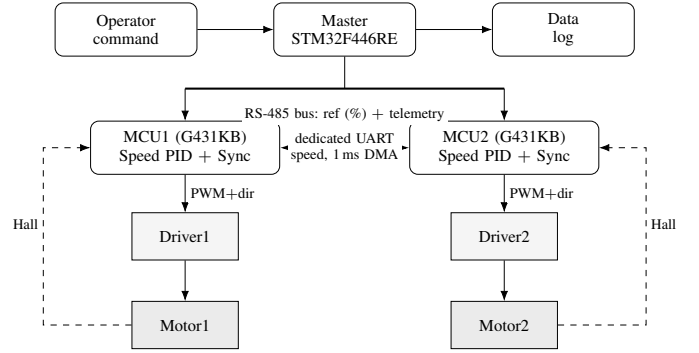


Fig. 2: Distributed control architecture. The master microcontroller broadcasts a speed reference over the RS-485 bus and logs both motor speeds. The two motor microcontrollers exchange their measured speeds over a dedicated 1 ms DMA UART link; each closes a local speed PID on its Hall-measured speed and adds a mirrored bilateral cross-coupled correction computed from both speeds, applied $\pm\frac{1}{2}$. The off-the-shelf controllers perform Hall commutation.

input-capture: the controller times the intervals between Hall edges to recover an instantaneous motor-shaft speed (with four pole pairs there are $6 \times 4 = 24$ Hall transitions per mechanical revolution). The microcontroller reads no current or voltage telemetry—current limiting is entirely internal to the BLDC controller—so the Hall-derived speed is the only feedback in the loop. There is no inertial sensor (IMU, gyroscope, or magnetometer) on the vehicle.

For data collection the master logs the two motor speeds it gathers over the bus; the logged quantities (per-motor speed, their difference, and the per-motor command) mirror the simulator's telemetry used for tuning. The yellow tape on the otherwise black track belts (Marker 1/Marker 2 in Fig. 1) is a qualitative visual aid that lets an observer see a track-speed difference during a run; it is not a measurement instrument. Quantitative inter-track synchronization error is therefore obtained from the Hall-logged speeds, while heading and path deviation are assessed from the vehicle's straight-line trajectory on the open-field test course.

D. Motor Mismatch

The two drives are nominally identical—the same motor, gearbox, and controller model on each side—but low-cost BLDC motors of this class vary from unit to unit in winding resistance and back-EMF constant, so at equal command one track runs persistently faster than the other. This unit-to-unit mismatch is a structured, sustained disturbance (not random noise) and is the dominant effect the synchronization layer must reject. For the offline tuning of Section IV this disturbance is represented by an identified BLDC model in which Motor 2 carries a -10% phase resistance and a -8% back-EMF/torque constant relative to Motor 1 (Table II); at equal duty this makes Motor 2 both faster (lower K_e) and slightly stronger in torque-per-duty, so it leads on acceleration and lags on deceleration. The mismatch of the deployed motors

TABLE I: Platform and Motor Specifications

Parameter	Value
<i>Vehicle</i>	
Type	Tracked skid-steer differential drive
Mass (as tested)	120 kg (75 carrier + 45 arm)
Dimensions (L×W×H)	1.5 × 0.75 × 0.5 m
Track type / running gear	Rubber belt; 2 idlers/side, rear sprocket
Track gauge B	1.2 m
Ground-contact length L	1.0 m
Drive sprocket radius r	0.20 m
Max track speed	11.3 km h ⁻¹ (at 3000 rpm)
<i>Motor and drivetrain (per side, identical)</i>	
Motor type	3-phase BLDC (auto-rickshaw), sensed
Gearbox	20:1 reduction
Rated voltage / power	48 V / 750 W
Rated / no-load current	15.6 A / ≈4 A
Max commanded speed	3000 rpm (motor shaft)
Pole pairs	4
Controller	Off-the-shelf 48 V/750 W BLDC, ≈30 A limit, ≈41.5 V UVLO
<i>Electronics and power</i>	
Motor MCU (×2)	STM32G431KB (Nucleo-32), 170 MHz
Master MCU	STM32F446RE
Sync link (MCU1↔MCU2)	Dedicated UART, DMA, 1 ms
Command / log bus	RS-485 (MAX485), UART 115 200 baud, custom framing, DMA + CRC [45]
Speed sensing	3 Hall lines/motor, timer input-capture
Battery	13S Li-ion, 48.1 V, 15 A h (shared)
Safety	Manual e-stop (all power); 20 A fuse/motor

TABLE II: Identified BLDC Model Parameters Used for Offline GA Tuning (Section IV)^a

Parameter	Motor 1	Motor 2
Phase resistance R (Ω)	0.150	0.135
Phase inductance L (mH)	0.5	0.5
Back-EMF const. K_e (V·s/rad)	0.0650	0.0598
Torque const. K_t (N·m/A)	0.0650	0.0598
Inertia J ($\times 10^{-4}$ kg·m ²)	4.8	4.8
Viscous damping B ($\times 10^{-3}$)	1.0	1.0
Coulomb friction T_c (N·m)	0.02	0.02

^aIdentified-model values used only for the offline GA tuning of Section IV. Motor 2 carries a -10% R and -8% K_e/K_t as a representative design-basis mismatch; the tuning model also uses a raised supply and an ≈ 3300 rpm speed normalization for numerical reachability. These are to be replaced with bench-measured values for the deployed motors.

is to be characterized on the bench and substituted for the modeled values.

IV. CONTROLLER DESIGN

The controller has two layers. Each motor runs a local speed PID with feedforward; on top, a bilateral cross-coupled term corrects the inter-motor speed difference. The two layers are realized in a *distributed* fashion — one microcontroller per motor — with the measured speeds exchanged over a dedicated inter-MCU link so each side can form the same synchronization correction (Section V); a separate master node distributes the speed command and logs data over a custom RS-485 bus whose design we present in [45]. All gains are obtained offline by a genetic algorithm (Section IV-C). Figure 3 shows the architecture; the commanded speed is first passed through a second-order rate limiter that produces a smooth reference r and its velocity $\dot{r}$ (peak rate $\dot{r}_{\max}$), which the feedforward terms use.

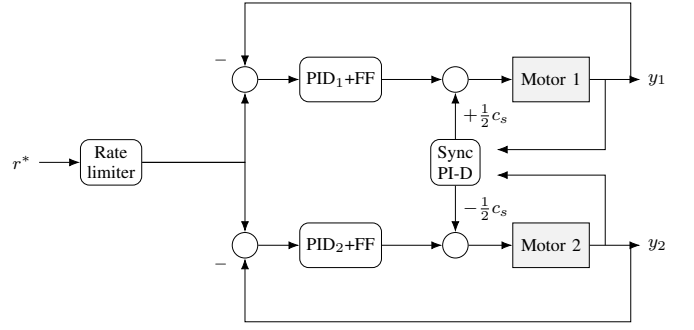


Fig. 3: Distributed bilateral cross-coupled control architecture. Each motor runs a local speed PID with feedforward on its own MCU; a synchronization PI-D acting on $y_2 - y_1$ injects $\pm \frac{1}{2} c_s$ into the two duty commands. The rate limiter shapes the speed reference r ; per-motor speed feedback returns to the summing junctions.

A. Per-Motor Speed PID with Feedforward

Each motor $i \in \{1, 2\}$ is regulated by a discrete PID acting on the normalized speed error

$$e_i = \text{clamp}\left(\frac{r - y_i}{y_{\max}}, -1, +1\right), \quad (1)$$

with y_i the measured speed and $y_{\max}$ a normalization speed. The loop uses derivative-on-measurement through a first-order filter and a clamped integrator:

$$u_i^{\text{PID}} = D_{\max} \left(K_p e_i + K_i \int e_i dt - K_d \dot{y}_i / y_{\max} \right), \quad (2)$$

$$\dot{y}_i \leftarrow \alpha \dot{y}_i + (1 - \alpha) (y_i - y_i^-) / \Delta t, \quad (3)$$

where $D_{\max} = 100\%$ duty, α is the filter coefficient and y_i^- the previous sample. The integrator is held in an *asymmetric* clamp $[-I^-, I^+]$ — a unipolar drive cannot brake, so little negative range is useful — and corrected by back-calculation anti-windup,

$$I_i \leftarrow \text{clamp}(I_i - K_b(u_i^{\text{raw}} - d_i), -I^-, I^+), \quad (4)$$

d_i being the saturated command (6). Two feedforward terms supply the steady and inertial duty so the integrator need not:

$$u_i^{\text{FF}} = K_v r + K_a \dot{r} \pm \frac{1}{2} \delta_{\text{ff}} \frac{r}{y_{\max}}, \quad (5)$$

the static-mismatch decoupling term taking $+$ for motor 1 and $-$ for motor 2 to pre-bias the two duties by the K_e gap. The duty applied to motor i is

$$d_i = \text{sat}_{[0,100]}(u_i^{\text{PID}} + u_i^{\text{FF}} + c_i), \quad (6)$$

with synchronization correction c_i defined next; u_i^{FF} and c_i are excluded from the anti-windup term (4) so the integrator reacts only to its own saturation. Constants: $\alpha = 0.30$, $[-I^-, I^+] = [-5, 100]\%$, $K_b = 0.04$. The anti-windup and filtered-derivative forms follow [46]–[48].

B. Bilateral Cross-Coupled Synchronization

The synchronization layer drives the inter-motor speed difference to zero. Using the same normalized, filtered PI-D form on the difference,

$$e_s = \text{clamp}\left(\frac{y_2 - y_1}{y_{\max}}, -1, 1\right), \quad c_s = D_{\max}\left(K_p^s e_s + K_i^s \int e_s dt - K_d^s \dot{e}_s\right), \quad (7)$$

the correction is applied *bilaterally* — split equally and oppositely between the drives — together with a differential-acceleration feedforward $K_{\text{ffd}}\dot{r}$ that pre-empts the mismatch-induced lead/lag on ramps:

$$c_1 = +\frac{1}{2}c_s + \frac{1}{2}K_{\text{ffd}}\dot{r}, \quad c_2 = -\frac{1}{2}c_s - \frac{1}{2}K_{\text{ffd}}\dot{r}. \quad (8)$$

For two motors this bilateral law is equivalent to Koren cross-coupled control [1], [2]: each drive is corrected by half of the pairwise speed error, so neither is a fixed master and a disturbance on either motor is rejected symmetrically [15]. Because $K_{\text{ffd}}\dot{r}$ vanishes at constant speed ($\dot{r} = 0$), it acts only during transients and leaves the steady-state difference untouched. The synchronization loop is executed faster than the per-motor loops; the loop rates and the inter-MCU speed exchange are detailed in Section V.

C. Offline Controller Tuning by Genetic Algorithm

The controller gains in (2)–(8) were obtained *offline* by a genetic algorithm (GA) that evaluates candidate gain sets against an identified model of the two motors before deployment.

The plant model is an averaged-value, trapezoidal six-step BLDC model evaluated separately for each motor with the parameters in Table I; the inter-motor mismatch (-10% R , -8% K_e) is reproduced so the GA optimizes against the same differential disturbance the vehicle exhibits. *[Parameters are the identified-model values; refresh after on-hardware identification.]*

The GA evolves an eleven-gene chromosome: the three Motor-1 PID gains, the three synchronization PID gains, the integrator preload, the differential-acceleration gain K_{ffd} , and three gains for a co-tuned Motor-2 loop so the slower motor is optimized in the synchronization context. Each candidate is evaluated by simulating the closed loop over four speed-step scenarios ($0 \rightarrow 3000$, $3000 \rightarrow 1500$, $1500 \rightarrow 2500$, $2500 \rightarrow 1000$ RPM) and scored by a fitness $f = 1/(1 + J)$, where the penalty J weights the worst-instant inter-motor deviation, the mean transient and steady-state synchronization error, and each motor's own tracking error; per-scenario penalties are combined by their mean so no scenario can be neglected. The population is evolved with simulated-binary crossover, bounded mutation with boundary escape, tournament selection, and multi-elitism, with diversity-driven re-initialization on stagnation [35], [37]; GA-based PID tuning for BLDC drives follows [6], [31].

Tuning proceeds in two phases (Fig. 4): Motor 2's speed loop is first optimized in isolation, after which the synchronization controller and Motor 1 are co-tuned with Motor 2 in the loop. In each phase the best fitness rises steeply within the first few tens of generations and then refines slowly onto a plateau, the co-tuned synchronization controller reaching

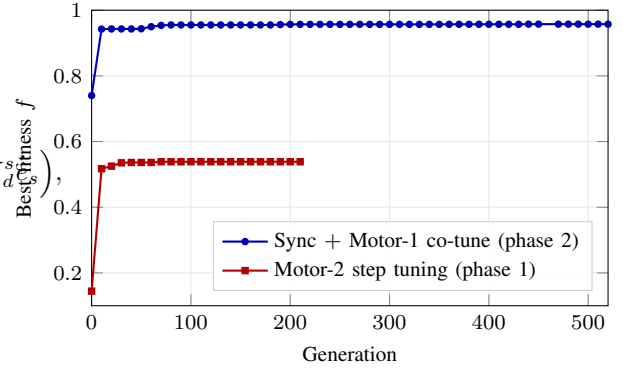


Fig. 4: Offline GA convergence for the fresh two-phase tuning run: best fitness versus generation. Phase 1 tunes Motor 2's speed loop in isolation; Phase 2 co-tunes the synchronization controller and Motor 1 with Motor 2 in the loop. Each phase optimizes its own objective, so the two curves are not directly comparable; both rise steeply within a few tens of generations and then refine onto a plateau.

TABLE III: GA-Tuned Controller Gains (offline; to be verified on hardware)

Loop	K_p	K_i	K_d
Motor 1 speed	2.9611	5.5944	0.00013
Motor 2 speed (sync ctx.)	2.1954	3.7638	0.00010
Synchronization	18.4639	37.8060	0.00809

Sync integrator preload = 0.0100
 Feedforward: $K_v = 0.0272$, $K_a = 2.0 \times 10^{-4}$,
 $\delta_{\text{ff}} = 8.0\%$, $K_{\text{ffd}} = -6.95 \times 10^{-4}$

$f \approx 0.96$. In closed-loop simulation with these gains the inter-motor speed difference settles to about 0.5 rpm ($< 0.02\%$ of full speed) at constant speed and peaks at roughly 5 rpm to 8 rpm during the commanded speed steps. These offline gains (Table III) seed the deployed controller and are then verified — and, if needed, lightly re-tuned — on the vehicle; the residual sim-to-real adjustment is reported in Section VI.

V. EMBEDDED IMPLEMENTATION

This section describes how the controller of Section IV runs on the vehicle: the firmware partition across microcontrollers, the two communication links, the loop timing, and the sensing, command, and logging chain.

A. Distributed Firmware Partition

Control is split across three STM32 microcontrollers. Each motor is driven by its own STM32G431KB (Nucleo-32, 170 MHz) that closes the local speed loop of (1)–(6) for its motor and computes the bilateral synchronization correction of (7)–(8). A master STM32F446RE converts the operator's motion command into a speed reference and distributes it. Because the synchronization law is symmetric, it is evaluated *identically and independently on both motor nodes* rather than on the master: each node receives the other motor's measured speed, forms the same correction c_s , and applies its half with the appropriate sign ($+\frac{1}{2}c_s$ on motor 1, $-\frac{1}{2}c_s$ on

motor 2). Neither motor node is a master of the other, and the F446RE never closes the synchronization loop—it only issues the common speed reference and records data.

The motor microcontrollers do not perform commutation. Each drives an off-the-shelf sensored BLDC controller (Table I) with a single PWM throttle line and a direction line; the controller performs the six-step Hall commutation and enforces its own current limit. The microcontroller's role is therefore to compute the duty command $d_i \in [0, 100]\%$ of (6) and present it to the controller's throttle input.

B. Communication Links

Two physically separate links serve two different needs (Fig. 2). The *synchronization* link is a dedicated point-to-point UART between the two motor microcontrollers: each transmits its latest measured speed to the other every 1 ms using DMA, so each node always has a fresh value of the peer speed for (7). Keeping this exchange on a private, deterministic channel is what lets the synchronization loop run at the rate the differential disturbance demands. The *command and telemetry* link is a separate RS-485 bus (MAX485 transceivers) on which the master broadcasts the speed reference (as a percentage of maximum speed) to both motor nodes and polls each for its measured speed for logging; it runs UART at 115 200 baud with custom framing, DMA transfers, and a per-packet CRC. The motor nodes are two slaves on this same multi-node bus, which also serves the vehicle's manipulator and sensors; the bus protocol, addressing, and measured timing are reported in our prior work [45]. Separating the two links means the slower, shared command/telemetry bus never sits in the synchronization path.

C. Control-Loop Timing

The firmware runs a two-rate cascade. The per-motor speed PID with feedforward updates every 5 ms (200 Hz), while the synchronization correction—and the duty command sent to each controller—updates every 1 ms (1 kHz), matched to the 1 ms inter-MCU speed exchange. The synchronization loop is run five times faster than the speed loop deliberately: the per-motor loops track a smooth, rate-limited reference and need only modest bandwidth, whereas the inter-motor *difference* contains components on the motors' electrical (L/R) timescale that a 5 ms loop would undersample; running the difference loop at 1 ms lets it counteract a developing lead/lag before the duty command saturates. The drive is unipolar, so each duty command is saturated to $[0, 100]\%$ (6); because the controller cannot brake, the integrator uses the asymmetric clamp and back-calculation anti-windup of (4). Commutation PWM is generated inside the off-the-shelf controllers and is not a parameter of the control firmware.

D. Sensing, Command, and Safety

Speed feedback is derived from each motor's three Hall-sensor lines. The microcontroller times the intervals between Hall edges by timer input-capture and converts them to motor-shaft speed; with four pole pairs there are $6 \times 4 = 24$

Hall transitions per mechanical revolution, giving fine speed resolution at low latency. No current or voltage telemetry is read by the microcontroller—current limiting is internal to the controller—so the Hall-derived speed is the only feedback in the loop, and there is no inertial sensor on the vehicle.

The command chain is: the operator issues a motion command (forward, reverse, or a turn); the master maps it to a speed reference as a percentage of maximum speed and broadcasts it over the RS-485 bus; each motor node converts the percentage to an RPM set-point (0–3000 rpm at the motor shaft) and tracks it with its local loop plus the synchronization correction. A straight-line command yields an equal reference to both motors—the case the synchronization layer is designed for—while a turn yields differential references. Safety is handled by a chassis-mounted manual emergency stop that disconnects all power and by a 20 A fuse in each motor circuit.

E. Data Logging

During each run the master gathers the two motor speeds over the RS-485 telemetry bus and logs them, together with their difference and the per-motor duty, at the control cadence; these logged signals are the quantitative measurement used for the results of Section VI (inter-track synchronization error is computed directly from the logged speeds). Because a single node (the master) timestamps and records all channels, the logged signals are mutually time-aligned without a separate synchronization step. The yellow track markers (Section III) serve only as an in-the-field visual check of the speed difference and are not used for quantitative measurement.

VI. RESULTS

[The data below are from closed-loop simulation of the controller against the identified two-motor plant (Section IV); on-vehicle field trials are the next step. Re-label as preliminary/sim accordingly.]

We evaluate the controller of Section IV in closed-loop simulation against the identified two-motor plant of Section III, comparing the full *synchronization-ON* controller (per-motor PID with feedforward, the bilateral cross-coupled sync loop, mismatch decoupling, and differential-acceleration feedforward) against a *synchronization-OFF* baseline in which each motor tracks the same speed command with its own independent PID and *no* inter-motor coupling or mismatch compensation. Both runs execute the identical four speed-step scenarios used for tuning ($0 \rightarrow 3000$, $3000 \rightarrow 1500$, $1500 \rightarrow 2500$, $2500 \rightarrow 1000$ rpm; 10 s each).

A. Metrics

From the logged motor speeds m_1, m_2 (motor-shaft rpm) we report: the *instantaneous synchronization error* $m_1 - m_2$; the *steady-state error* (mean $|m_1 - m_2|$ over the settled second half of each scenario); the *peak transient error* (maximum $|m_1 - m_2|$); the *synchronization-error RMSE* over the run; and the *heading drift*, obtained by integrating the speed difference through the differential-drive kinematics of Section III,

$$\psi(t) = \int_0^t \frac{(m_1 - m_2) \frac{2\pi}{60} r}{g B} d\tau, \quad (9)$$

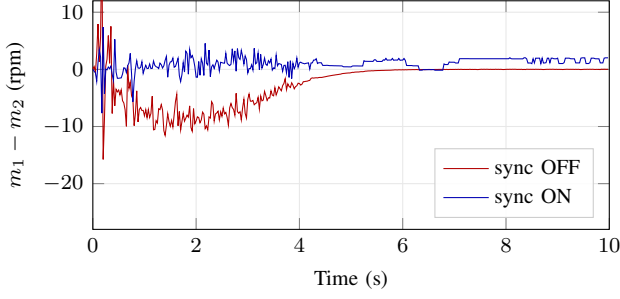


Fig. 5: Straight-line start-up to cruise ($0 \rightarrow 3000$ rpm): inter-motor speed difference $m_1 - m_2$, synchronization ON vs OFF. Synchronization cuts the start-up transient from ≈ 24 rpm to < 8 rpm.

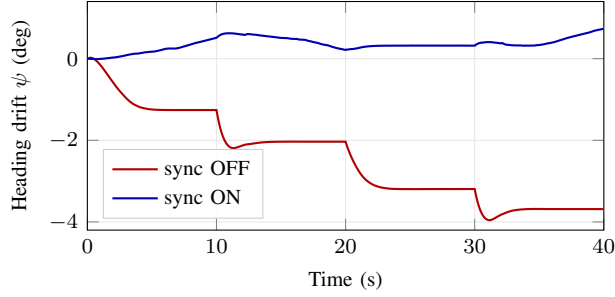


Fig. 6: Accumulated heading drift over the four-maneuver run (9). Synchronization reduces the drift from $\approx 3.7^\circ$ to $\approx 0.7^\circ$.

with gear ratio $g = 20$, sprocket radius $r = 0.20$ m, and track gauge $B = 1.20$ m; ψ is reported in degrees.

B. Straight-Line Tracking

Figure 5 shows the inter-motor speed difference during the $0 \rightarrow 3000$ rpm start-up and cruise. Both motors track the command (their speeds overlap and are omitted); the difference is the quantity of interest. Without synchronization the mismatch produces a start-up transient of about 24 rpm—Motor 2, with the lower back-EMF constant, accelerates faster—which the bilateral sync loop limits to under 8 rpm. As the two independent integrators settle, the synchronization-OFF difference decays toward zero at constant speed (both loops reach the set-point), whereas the synchronization-ON case retains a small (≈ 1.4 rpm) steady ripple from the fast multi-rate sync loop.

C. Heading Drift

The transient differences integrate, through (9), into a *persistent* heading offset that does not wash out when the speeds re-converge. Figure 6 accumulates the heading drift across the full four-maneuver sequence: without synchronization the vehicle drifts $\approx 3.7^\circ$ off heading, versus $\approx 0.7^\circ$ with synchronization—a roughly five-fold reduction. This is the operationally important result, since it is the heading error, not the instantaneous speed match, that steers the vehicle off its intended path.

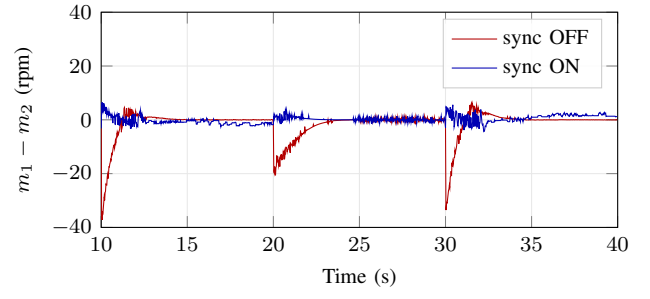


Fig. 7: Synchronization error during the speed-step maneuvers ($3000 \rightarrow 1500 \rightarrow 2500 \rightarrow 1000$ rpm). Peak transient difference falls from 25–37 rpm (OFF) to 5–8 rpm (ON).

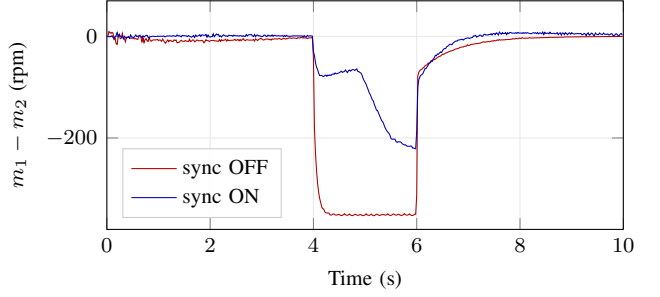


Fig. 8: Single-track load disturbance (1.0 N m on Motor 1, $t = 4$ – 6 s): inter-motor speed difference, synchronization ON vs OFF. The loaded motor saturates; synchronization slows the other motor to match, cutting the sustained difference from ~ 344 to ~ 125 rpm.

D. Speed-Step Response

Figure 7 shows the synchronization error during the three speed-step maneuvers ($t = 10$ – 40 s). Each commanded step excites a transient mismatch; without synchronization the peaks reach 25–37 rpm, whereas the sync loop holds them to 5–8 rpm.

E. Disturbance Rejection

Figure 8 applies a single-track load disturbance (1.0 N m on Motor 1 only) for 2 s during cruise at 3000 rpm. At this speed the loaded motor reaches its duty ceiling and cannot hold speed on its own, so this is the regime an independent controller cannot handle. Without synchronization, Motor 1 sags about 344 rpm below Motor 2 for the *entire* load—a sustained difference, not a transient—and the heading kicks by $\approx 35^\circ$. With synchronization the loop slows Motor 2 to follow the loaded motor, cutting the sustained difference to 125 rpm and the heading excursion to $\approx 12^\circ$ (a $\sim 65\%$ reduction). This is the regime where synchronization helps at *steady state*, not only in transients: an independent controller cannot coordinate against a one-sided load it lacks the authority to reject, whereas the cross-coupled loop trades a little common-mode speed to keep the two drives together.

F. Summary and Discussion

Table IV summarizes the comparison. Synchronization reduces the peak transient inter-motor error by $\approx 79\%$, the

TABLE IV: Synchronization ON vs OFF (closed-loop simulation)

Metric	Sync OFF	Sync ON	Improvement
<i>Speed-step run (no disturbance)</i>			
Peak transient $ m_1 - m_2 $ (rpm)	37.2	7.7	79%
Sync-error RMSE (rpm)	5.51	1.47	73%
Heading drift, full run (deg)	3.69	0.73	80%
Steady-state $ m_1 - m_2 $ (rpm)	0.13	1.17	− ^a
<i>Single-track load disturbance (1.0 N m)</i>			
Sustained $ m_1 - m_2 $ (rpm)	344	125	64%
Heading kick (deg)	35.5	12.3	65%

^aOn a constant set-point both independent integrators reach the target, so the steady-state speed match is already tight without synchronization; the small ON value is the multi-rate sync ripple.

synchronization-error RMSE by $\approx 73\%$, and the accumulated heading drift by $\approx 80\%$. The steady-state speed difference is small in both cases: on a *constant* command two independent integrators each reach the set-point, so synchronization does not improve—and, through the small multi-rate ripple, marginally increases—the steady-state speed match. The value of synchronization is therefore concentrated in the transients and, crucially, in the heading drift those transients leave behind. Under a sustained one-sided disturbance (Fig. 8) the benefit extends to steady state: synchronization cuts the held inter-motor difference by $\sim 64\%$ and the heading excursion by $\sim 65\%$. These figures are from closed-loop simulation against the identified plant; on-vehicle field trials are the next step and the natural sim-to-real comparison point.

VII. CONCLUSION AND FUTURE WORK

[TODO: Recap: a GA-tuned bilateral cross-coupled speed-synchronization controller was implemented on a dual-MCU architecture and validated on a real tracked differential-drive vehicle with two mismatched BLDC motors, reducing inter-wheel speed error and heading drift by [TODO: X%] versus an independent-PID baseline.]

[TODO: Future work: more aggressive maneuvers and varied terrain; online/adaptive re-tuning; extension to more than two drives; comparison against a robust (e.g. SMC/ADRC) synchronization law on the same hardware.]

ACKNOWLEDGMENT

[TODO: Optional — funding, lab, people.]

REFERENCES

- [1] Y. Koren, “Cross-Coupled Biaxial Computer Control for Manufacturing Systems,” *Journal of Dynamic Systems, Measurement, and Control (ASME)*, 1980. [Online]. Available: <https://asmedigitalcollection.asme.org/dynamicsystems/article/102/4/265/399976>
- [2] L. Feng, Y. Koren, and J. Borenstein, “Cross-coupling motion controller for mobile robots,” *IEEE Control Systems Magazine*, 1993. [Online]. Available: <https://doi.org/10.1109/37.248002>
- [3] F. Niu, K. Sun, S. Huang, Y. Hu, D. Liang, and Y. Fang, “A Review on Multimotor Synchronous Control Methods,” *IEEE Transactions on Transportation Electrification*, vol. 9, no. 1, 2023. [Online]. Available: <https://doi.org/10.1109/TTE.2022.3168647>
- [4] Z. Miao, Y. He, H. Li, and Y. Wang, “Dual-PMSM speed synchronization control based on unified prediction model,” *Journal of Measurement Science and Instrumentation*, 2023. [Online]. Available: <https://www.sciopen.com/article/10.62756/jmsi.1674-8042.2023036>
- [5] B. Li, H. Chai, and L. Hou, “Fixed-time leader-following speed consensus control for multi-PMSMs based on multi-agent systems consensus,” *Scientific Reports*, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-025-18061-3>
- [6] M. H. Mahmood, O. F. Ali, M. J. Mnati, B. M. Sabbar, and A. Van Den Bossche, “Intelligent PID Parameter Tuning for BLDC Motors by Using Genetic Algorithms,” *Journal Européen des Systèmes Automatisés (JESA)*, vol. 58, no. 7, 2025. [Online]. Available: <https://doi.org/10.18280/jesa.580714>
- [7] R. C. Garimella *et al.*, “Modeling and PID control of the brushless DC motor with the help of Genetic Algorithm,” in *2012 IEEE Int. Conf. on Power, Control and Embedded Systems (ICPCEs)*, 2012, author list + DOI to verify (page 1 was a ResearchGate cover). [Online]. Available: <https://ieeexplore.ieee.org/document/6216186/>
- [8] R. Megalingam, S. Manoharan, S. Mohandas, and A. Kamalasanan, “Design and Development of a Controller PCB for a Dual BLDC-Based Differential Drive Rover,” in *Lecture Notes in Networks and Systems, Springer*, 2025. [Online]. Available: https://doi.org/10.1007/978-981-96-0228-5_10
- [9] M. Reda, A. Onsy, A. Haikal, and A. Ghanbari, “Motor Speed Control of Four-wheel Differential Drive Robots Using a New Hybrid Moth-flame Particle Swarm Optimization (MFPSO) Algorithm,” *Journal of Intelligent & Robotic Systems*, 2025. [Online]. Available: <https://doi.org/10.1007/s10846-025-02228-1>
- [10] V. Le, D. Tran, and T. Nhat, “Synchronous Control of Dual Motor with Master-Slave and Cross-Coupling Methods,” in *Lecture Notes in Networks and Systems (Computational Intelligence Methods for Green Technology and Sustainable Development)*, Springer, 2022. [Online]. Available: https://doi.org/10.1007/978-3-031-19694-2_52
- [11] X. Zhang, H. Hu, H. Wang, and Z. Wang, “Overview of position synchronous control technology for multi-motor system,” *Systems Science & Control Engineering*, 2024. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/21642583.2024.2427074>
- [12] H. Huang, Q. Tu, C. Jiang, L. Li, P. Li, and H. Zhang, “Dual motor drive vehicle speed synchronization and coordination control strategy,” in *AIP Conference Proceedings*, vol. 1955, 2018, p. 040005. [Online]. Available: <https://doi.org/10.1063/1.5033669>
- [13] V. Jerković Štil, T. Varga, T. Benšić, and M. Barukčić, “A Survey of Fuzzy Algorithms Used in Multi-Motor Systems Control,” *Electronics (MDPI)*, vol. 9, no. 11, p. 1788, 2020. [Online]. Available: <https://doi.org/10.3390/electronics9111788>
- [14] W. Chen, J. Liang, and T. Shi, “Speed Synchronous Control of Multiple Permanent Magnet Synchronous Motors Based on an Improved Cross-Coupling Structure,” *Energies (MDPI)*, 2018. [Online]. Available: <https://doi.org/10.3390/en11020282>
- [15] T. Shi, H. Liu, Q. Geng, and C. Xia, “Improved relative coupling control structure for multi-motor speed synchronous driving system,” *IET Electric Power Applications*, 2016. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-epa.2015.0515>
- [16] Y. Mu, L. Qi, M. Sun, and W. Han, “An Improved Deviation Coupling Control Method for Speed Synchronization of Multi-Motor Systems,” *Applied Sciences (MDPI)*, 2024. [Online]. Available: <https://www.mdpi.com/2076-3417/14/12/5300>
- [17] L.-B. Li, L.-L. Sun, S.-Z. Zhang, and Q.-Q. Yang, “Speed tracking and synchronization of multiple motors using ring coupling control and adaptive sliding mode control,” *ISA Transactions*, 2015. [Online]. Available: <https://doi.org/10.1016/j.isatra.2015.07.010>
- [18] S. Lin, Y. Cai, B. Yang, and W. Zhang, “Electrical line-shafting control for motor speed synchronisation using sliding mode controller and disturbance observer,” *IET Control Theory & Applications*, 2017. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/iet-cta.2016.0924>
- [19] H. Huang, Q. Tu, C. Jiang, M. Pan, and C. Zhu, “An Electronic Line-Shafting Control Strategy Based on Sliding Mode Observer for Distributed Driving Electric Vehicles,” *IEEE Access*, 2021. [Online]. Available: <https://doi.org/10.1109/ACCESS.2021.3062829>
- [20] M. Valenzuela and R. Lorenz, “Electronic line-shafting control for paper machine drives,” *IEEE Transactions on Industry Applications*, 2001. [Online]. Available: <https://ieeexplore.ieee.org/document/903141>
- [21] D. Jung and S. Kim, “A Passive Decomposition Based Robust Synchronous Motion Control of Multi-Motors and Experimental Verification,” *Sensors (MDPI)*, 2023. [Online]. Available: <https://doi.org/10.3390/s23177603>
- [22] C. Zhu, Q. Tu, C. Jiang, M. Pan, and H. Huang, “A Cross Coupling Control Strategy for Dual-Motor Speed Synchronous System Based on Second Order Global Fast Terminal Sliding

- Mode Control,” *IEEE Access*, 2020. [Online]. Available: <https://doi.org/10.1109/ACCESS.2020.3042256>
- [23] K. Wu, Y. Zhang, Y. Qi, and W. Shi, “Control Strategy for Improving Dual Motor Synchronization Accuracy: Cross Coupling Method Based on Improved Global Fast Terminal Sliding Mode Control and Disturbance Observer,” *Applied Sciences (MDPI)*, 2025. [Online]. Available: <https://doi.org/10.3390/app15041915>
 - [24] C. Lan, H. Wang, X. Deng, X.-F. Zhang, and H. Song, “Multi-motor position synchronization control method based on non-singular fast terminal sliding mode control,” *PLOS ONE*, 2023. [Online]. Available: <https://doi.org/10.1371/journal.pone.0281721>
 - [25] Z. Kuang, H. Gao, and M. Tomizuka, “Precise Linear-Motor Synchronization Control Via Cross-Coupled Second-Order Discrete-Time Fractional-Order Sliding Mode,” *IEEE/ASME Transactions on Mechatronics*, 2020, template dated 2019; published 2020 — confirm year. [Online]. Available: <https://doi.org/10.1109/TMECH.2020.3019883>
 - [26] Y. Dai, L. Zhang, D. Xu, Q. Chen, and X.-G. Yan, “Anti-Disturbance Cooperative Fuzzy Tracking Control of Multi-PMSMs Low-Speed Urban Rail Traction Systems,” *IEEE Transactions on Transportation Electrification*, 2022. [Online]. Available: <https://doi.org/10.1109/TTE.2021.3133316>
 - [27] Z. Liu, W. Lin, X. Yu, J. J. Rodríguez-Andina, and H. Gao, “Approximation-Free Robust Synchronization Control for Dual-Linear-Motors-Driven Systems With Uncertainties and Disturbances,” *IEEE Transactions on Industrial Electronics*, 2022. [Online]. Available: <https://doi.org/10.1109/TIE.2021.3137619>
 - [28] K. Zhang, L. Wang, and X. Fang, “Robust decoupled synchronization control of dual-motor servo systems by mechanical parameter identification,” *IET Power Electronics*, 2022. [Online]. Available: <https://doi.org/10.1049/pel2.12320>
 - [29] C. Zhang, M. Niu, J. He, K. Zhao, H. Wu, and M. Zhang, “Robust synchronous control of multi-motor integrated with artificial potential field and sliding mode variable structure,” *IEEE Access*, 2016. [Online]. Available: <https://doi.org/10.1109/ACCESS.2016.2636224>
 - [30] S. Zou and W. Zhao, “Synchronization and stability control of dual-motor intelligent steer-by-wire vehicle,” *Mechanical Systems and Signal Processing*, 2020. [Online]. Available: <https://doi.org/10.1016/j.ymssp.2020.106925>
 - [31] M. A. Ibrahim, A. K. Mahmood, and N. S. Sultan, “Optimal PID controller of a brushless DC motor using genetic algorithm,” *International Journal of Power Electronics and Drive Systems (IJPEDS)*, vol. 10, no. 2, pp. 822–830, 2019. [Online]. Available: <https://doi.org/10.11591/ijpeds.v10.i2.pp822-830>
 - [32] H. S. Dakheel, Z. B. Abdullah, and S. W. Shneen, “Advanced optimal GA-PID controller for BLDC motor,” *Bulletin of Electrical Engineering and Informatics*, vol. 12, no. 4, pp. 2077–2086, 2023. [Online]. Available: <https://doi.org/10.11591/eei.v12i4.4649>
 - [33] S. A. Deraz, “Genetic Tuned PID Controller Based Speed Control of DC Motor Drive,” *International Journal of Engineering Trends and Technology (IJETT)*, vol. 17, no. 1, 2014. [Online]. Available: <https://doi.org/10.14445/22315381/IJETT-V17P219>
 - [34] M. A. N. Abed, N. M. Hashim, Y. Nasar, A. O. Hanfesh, and A. A. R. Altahir, “Genetic Algorithm-Tuned PID for Sensorless PMSM Speed Control Using a Second-Order Sliding Mode Observer,” *Journal of Robotics and Control (JRC)*, vol. 6, no. 5, 2025. [Online]. Available: <https://journal.umy.ac.id/index.php/jrc/article/view/27013>
 - [35] S. B. Joseph, E. G. Dada, A. Abidemi, D. O. Oyewola, and B. M. Khammas, “Metaheuristic algorithms for PID controller parameters tuning: review, approaches and open problems,” *Heliyon*, 2022. [Online]. Available: <https://doi.org/10.1016/j.heliyon.2022.e09399>
 - [36] F. Shafique, M. S. Fakhra, A. Rasool, S. A. R. Kashif, and M. Suhail, “Analyzing the performance of metaheuristic algorithms in speed control of brushless DC motor: Implementation and statistical comparison,” *PLOS ONE*, 2024. [Online]. Available: <https://doi.org/10.1371/journal.pone.0310080>
 - [37] A. Hassanat, K. Almohammadi, E. Alkafaween, E. Abunawas, A. Hammouri, and V. B. S. Prasath, “Choosing Mutation and Crossover Ratios for Genetic Algorithms — A Review with a New Dynamic Approach,” *Information (MDPI)*, 2019. [Online]. Available: <https://www.mdpi.com/2078-2489/10/12/390>
 - [38] V. Ha, T. Thuong, and L. Truc, “Trajectory tracking control based on genetic algorithm and proportional integral derivative controller for two-wheel mobile robot,” *Bulletin of Electrical Engineering and Informatics*, 2024. [Online]. Available: <https://doi.org/10.11591/eei.v13i4.7847>
 - [39] H. Wang, H. Du, Q. Cui, P. Liu, and X. Ma, “A multi-motor speed synchronization control enhanced by artificial bee colony algorithm,” *Measurement and Control*, 2023. [Online]. Available: <https://doi.org/10.1177/00202940221122160>
 - [40] A. Ashraf, H. Ahsan, and M. Arshad, “Motor Speed Synchronization of Mobile Robot Using PI Controller,” in *2021 Int. Conf. on Digital Futures and Transformative Technologies (ICoDT2)*, 2021. [Online]. Available: <https://doi.org/10.1109/ICoDT252288.2021.9441518>
 - [41] Y.-B. Choi, S.-H. Lee, and D.-H. Jung, “Controller Design Based on a Synchronous Error Compensator for Wheeled Cross-coupled Systems,” *International Journal of Control, Automation and Systems*, 2024. [Online]. Available: <https://doi.org/10.1007/s12555-023-0687-x>
 - [42] Y. L. Karnavas, A. S. Topalidis, and M. Drakaki, “Development and Implementation of a Low Cost μ C-Based Brushless DC Motor Sensorless Controller: A Practical Analysis of Hardware and Software Aspects,” *Electronics (MDPI)*, 2019. [Online]. Available: <https://www.mdpi.com/2079-9292/8/12/1456>
 - [43] C. Zhao and Z. Hua, “Design of Motor Speed Control System Based on STM32 Microcontroller,” in *2022 IEEE Int. Conf. on Computation, Big-Data and Engineering (ICCBDE)*, 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9888225/>
 - [44] S. Amornwongpeeti, M. Ekpanyapong, N. Chayopitak, J. L. Monteiro, J. S. Martins, and J. L. Afonso, “A Single Chip FPGA-Based Cross-Coupling Multi-Motor Drive System,” *IEICE Electronics Express*, vol. 12, no. 13, 2015. [Online]. Available: <https://doi.org/10.1587/ele.12.20150383>
 - [45] M. S. Tanvir, F. Ahmed, M. M. Islam, M. Islam, M. Hasan, and R. U. Rashid, “Optimizing Master-Slave Broadcast Communication in Multi-Node Network Using RS485 Standard,” in *2024 27th International Conference on Computer and Information Technology (ICCIIT)*. Cox’s Bazar, Bangladesh: IEEE, 2024, iEEE Xplore document 11022430; DOI to verify. [Online]. Available: <https://ieeexplore.ieee.org/document/11022430>
 - [46] K. Astrom and T. Hagglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC: Instrument Society of America (ISA), 1995. [Online]. Available: https://books.google.com/books/about/PID_Collectors.html?id=FsyhngEACAAJ
 - [47] C. Bohn and D. Atherton, “An Analysis Package Comparing PID Anti-Windup Strategies,” *IEEE Control Systems Magazine*, vol. 15, no. 2, pp. 34–40, 1995. [Online]. Available: <https://ieeexplore.ieee.org/document/375281/>
 - [48] V. Romero Segovia, T. Hagglund, and K. Astrom, “Measurement Noise Filtering for Common PID Tuning Rules,” *Control Engineering Practice*, vol. 32, pp. 43–63, 2014. [Online]. Available: <https://doi.org/10.1016/j.conengprac.2014.07.005>